StimaSaidizi

AI-Assisted Electricity Customer Support and Internal Analytics Prototype

Energy & Manufacturing · Kenya

Student: Leon Changara Chemwor Odari

Project: IBM Phase 3 Cornerstone Project
Pathway: Product Development
Project type: Solo project
Report status: Final implementation report
Date: September 2026

Demonstration notice. StimaSaidizi is an educational prototype using synthetic customer, billing, prepaid-token, outage and support-case data. It is not an official Kenya Power service and is not connected to live utility operational systems.

Executive Summary

StimaSaidizi is an AI-assisted customer-support and service-intelligence prototype designed for a Kenyan electricity-utility context. The project addresses recurring customer-service enquiries involving power outages and faults, prepaid-token transactions, billing questions, and support-case management. It also provides an internal analytics workspace, Mkurugenzi, for support managers to interrogate deterministic operational metrics and unresolved cases using natural language.

The project began with an IBM-centred architecture using watsonx.ai and Watson Orchestrate, consistent with the original Cornerstone project plan. The application logic was intentionally separated into modular Python services and FastAPI routes. When platform and deployment constraints affected the original orchestration path, the conversational layer was migrated to Google Agent Development Kit (ADK), with LiteLLM and Groq providing model access. The core service layer, synthetic datasets, tests and functional objectives were retained.

The final prototype is delivered through a Streamlit interface with two connected workspaces: Customer Support and Mkurugenzi - Service Operations. The customer agent invokes deterministic tools for customer, outage, token, billing and case-management operations. Mkurugenzi consumes metrics calculated by the analytics service rather than relying on the language model to calculate operational figures. This design keeps application data and service functions as the source of truth while using AI for interaction, explanation and summarisation.

Automated service testing completed successfully with 13 tests passing. Live validation demonstrated prepaid-token lookup, billing support and case creation, outage handling, management analytics, and visibility of a newly created customer case within the internal analytics workflow. The application was containerised and locally validated with Docker, and the final demonstration was deployed on Streamlit Community Cloud.

Contents

1 Introduction and Project Context **1**
- 1.1 Project background . . . 1
- 1.2 Problem statement . . . 1
- 1.3 Project objectives . . . 1
- 1.4 Scope . . . 1

2 Requirements and System Design **1**
- 2.1 Two-workspace product model . . . 2
- 2.2 Functional requirements . . . 2
- 2.3 Data model . . . 2

3 System Architecture **3**
- 3.1 End-to-end workflow . . . 3
- 3.2 Layered architecture . . . 3

4 Implementation **4**
- 4.1 Deterministic service layer . . . 4
- 4.2 Customer Support agent . . . 4
- 4.3 Mkurugenzi . . . 4
- 4.4 Streamlit interface . . . 5

5 Architecture Evolution **6**
- 5.1 Original IBM design . . . 6
- 5.2 Migration of the orchestration layer . . . 6

6 Testing and Validation **7**
6.1 Automated service tests . . . 8
6.2 Live customer-support validation . . . 8
6.3 Mkurugenzi validation . . . 8
6.4 Cross-feature validation . . . 9

7 Deployment **9**
7.1 Container validation . . . 9
7.2 Cloud deployment path . . . 9

8 Security, Governance and Responsible AI **10**
8.1 Implemented safeguards . . . 10
8.2 Production requirements . . . 10

9 Results and Discussion **10**

10 Limitations and Future Development **10**
10.1 Current limitations . . . 11
10.2 Future work . . . 11

11 Conclusion **11**

A Repository Structure **11**

B Demonstration Script **12**

C Project Artifacts **12**

D Source Basis for this Report **12**

1 Introduction and Project Context

1.1 Project background

Electricity customer-support environments routinely handle repetitive enquiries that require access to operational records rather than free-form generation. Typical examples include whether an area has a recorded outage, whether a prepaid-token payment succeeded, whether the corresponding token has been issued, the status of a bill, and whether an unresolved issue has been recorded for follow-up.

StimaSaidizi was developed as a solo IBM Phase 3 Cornerstone Project under the Product Development pathway in the Energy & Manufacturing sector. The working frame established in the original project plan was an AI-assisted customer-support and service-intelligence prototype for a Kenyan electricity utility. The planned system deliberately used synthetic data and did not attempt to replace existing utility customer-service infrastructure or connect to live Kenya Power systems.

1.2 Problem statement

A generic chatbot can produce fluent responses but may fabricate operational details when it does not have a reliable source of truth. In an electricity-support setting, fabricated outage restoration times, token states, billing amounts or field-service actions would make the system unreliable. The project therefore focuses on combining conversational AI with deterministic service functions and structured support data.

A second problem is operational visibility. Customer interactions are more useful to a service organisation when they also produce structured records that can be analysed. StimaSaidizi connects the customer-facing workflow to an internal analytics workflow so that created and escalated cases contribute to management-level service information.

1.3 Project objectives

The final implementation pursues the following objectives:

- Provide a customer-facing assistant for outage/fault, prepaid-token and billing enquiries.
- Ground operational answers in synthetic service records rather than model-generated facts.
- Create and escalate support cases through deterministic backend functions.
- Provide an internal analytics assistant for support metrics and unresolved-case analysis.
- Demonstrate a shared data flow between Customer Support and Internal Analytics.
- Preserve safe behaviour when requested operational information is unavailable.
- Deliver a usable web-based prototype suitable for Cornerstone demonstration.

1.4 Scope

The prototype supports English by default and includes instructions for Kiswahili interaction when explicitly requested. It uses synthetic JSON datasets representing customers, outages, prepaid-token transactions, bills and support cases. It does not process real payments, perform real technician dispatch, authenticate production customers, or access live utility infrastructure.

2 Requirements and System Design

2.1 Two-workspace product model

The user experience is divided into two connected workspaces. Customer Support handles service enquiries and case management, while Mkurugenzi provides analytics and summaries for support managers. Figure 1 illustrates this separation.

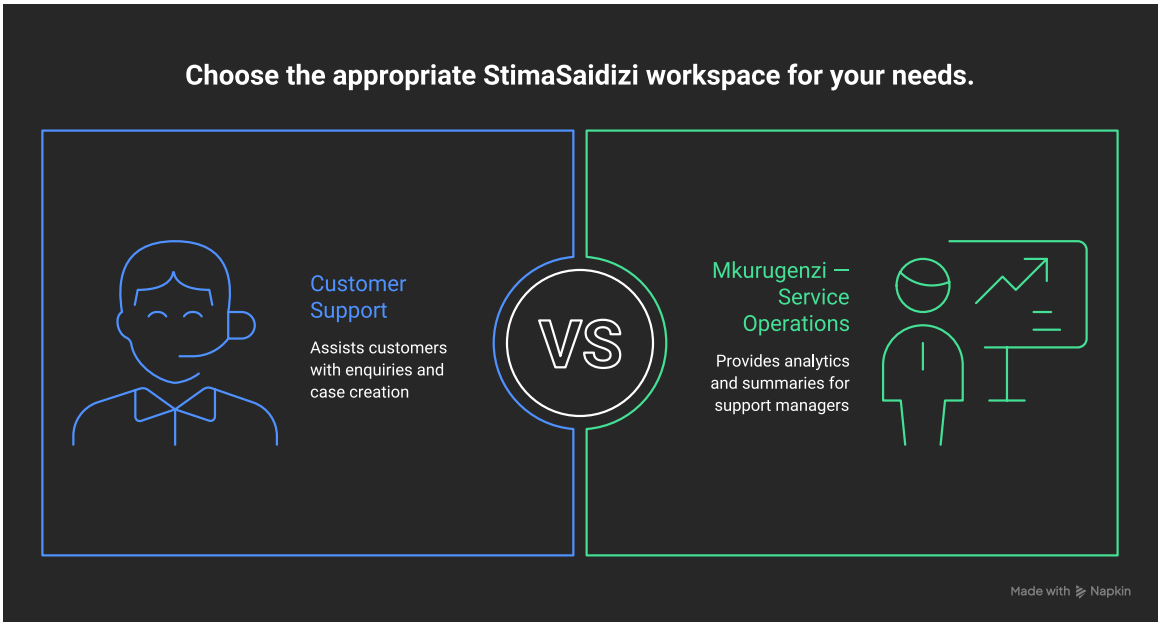


Figure 1: StimaSaidizi workspace model: Customer Support and Mkurugenzi - Service Operations.

2.2 Functional requirements

Capability	Required behaviour
Customer lookup	Retrieve a matching synthetic customer record when required by a workflow.
Outage support	Check recorded active outage information by area and avoid inventing outage causes or restoration estimates.
Prepaid-token support	Retrieve a transaction and distinguish payment status from token-issuance status.
Billing support	Retrieve only billing information available in the synthetic dataset.
Case creation	Create a structured support case and report a case ID only after successful backend creation.
Case escalation	Change case state through the case service and confirm escalation only after a successful tool result.
Internal analytics	Calculate support metrics deterministically and expose unresolved cases to Mkurugenzi.
Grounded conversation	Use the language model for interaction and explanation while treating tools and data as the operational source of truth.

2.3 Data model

The implemented data layer uses synthetic JSON records. The main entities are Customer, SupportCase, Outage, TokenTransaction and Bill. The original project plan also proposed an Interaction entity for persistent conversation events; the final prototype instead uses runtime conversational sessions and does not implement a production interaction database.

Entity	Representative fields
Customer	customer_id, name, county, area, location_class, preferred_language, customer_type, meter/account reference
SupportCase	case_id, customer_id, category, channel, language, timestamps, status, resolution time, escalation reason, summary
Outage	outage_id, area, county, start time, estimated restoration, status, outage type
TokenTransaction	transaction_id, customer_id, amount, transaction time, payment status, token status
Bill	bill_id, customer_id, billing period, amount due, due date, status

Table 2: Core synthetic data entities used by the prototype.

3 System Architecture

3.1 End-to-end workflow

Figure 2 shows the implemented flow from customer enquiry to service tools, synthetic data, analytics and the manager-facing Mkurugenzi workspace.

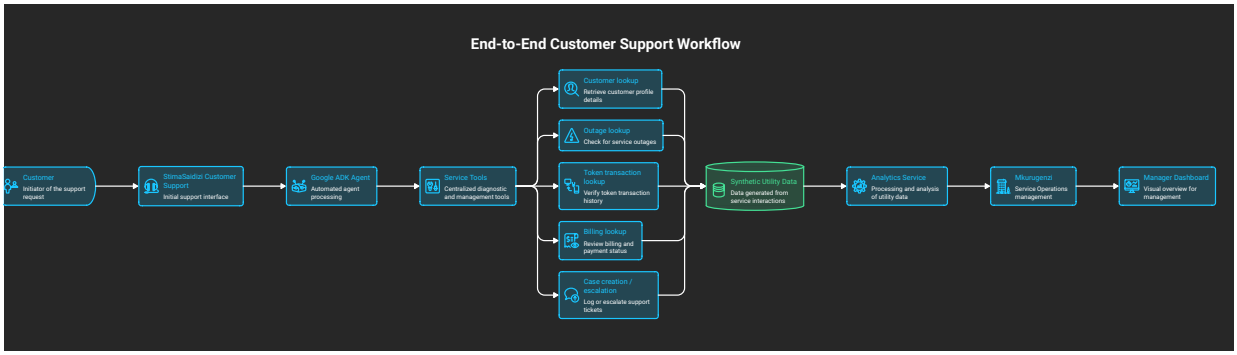


Figure 2: End-to-end StimaSaidizi customer-support and service-operations workflow.

The customer-facing and management features share the same service and data layer. A typical request proceeds through the following sequence:

1. The customer submits an enquiry through the Streamlit interface.
2. The StimaSaidizi ADK agent identifies the supported request and gathers any required identifier or area.
3. The agent invokes a deterministic service tool.
4. The tool retrieves or modifies synthetic application data.
5. The agent explains the tool result without inventing missing operational information.
6. Where necessary, a support case is created or escalated.
7. The analytics service calculates metrics from support-case records.
8. Mkurugenzi presents grounded operational summaries to the manager.

3.2 Layered architecture

The implementation is separated into presentation, agent/orchestration, application-service, API and data layers. This modularity was important because it allowed the orchestration technology to

change without redesigning the core service logic.

Layer	Technology	Role
Presentation	Streamlit	Customer Support, Manager Dashboard and conversational UI.
Agent framework	Google ADK	Agent lifecycle, tool invocation and conversational orchestration.
Model access	LiteLLM + Groq	LLM abstraction and GPT-OSS model access.
Application logic	Python services	Deterministic customer, outage, token, billing, case and analytics functions.
API	FastAPI	Modular HTTP routes and API-key protected backend interface.
Data	JSON	Synthetic MVP records for utility-support scenarios.
Testing	pytest	Deterministic service validation.
Deployment	Streamlit Community Cloud	Public demonstration hosting.
Containerisation	Docker	Reproducible container configuration and local deployment validation.

Table 3: Implemented technology stack.

4 Implementation

4.1 Deterministic service layer

The service layer forms the operational source of truth. `customer_service.py` retrieves customer records; `outage_service.py` checks active outage records and determines whether a stored restoration estimate has expired; `token_service.py` retrieves prepaid-token transaction status; `billing_service.py` retrieves bill records; `case_service.py` creates and escalates support cases; and `analytics_service.py` calculates support metrics and unresolved-case information.

This separation is intentional. The language model is not asked to invent or calculate operational state when a deterministic service can provide it. The same design also makes the core application independently testable.

4.2 Customer Support agent

The Customer Support agent is configured as an AI-assisted electricity customer-support assistant for the synthetic Kenyan utility context. Its registered tools wrap the deterministic services for customer lookup, outage verification, token verification, bill retrieval, case creation and case escalation.

Important behavioural rules include asking for required identifiers when missing, distinguishing payment success from token issuance, avoiding invented restoration times, and reporting case identifiers only after successful backend creation. English is the default response language, while Kiswahili can be used when requested.

4.3 Mkurugenzi

Mkurugenzi is the internal service-operations assistant. Its toolset is intentionally narrower than the customer agent. It receives a deterministic analytics summary and unresolved-case records. Metrics include total cases, category and status distributions, resolution and escalation rates, average resolution time, location-class resolution time and the most common enquiry category.

The prompt instructs Mkurugenzi not to infer improvement, deterioration or causal explanations when the dataset does not support those conclusions. It also treats zero or missing values carefully.

For example, a rural average resolution time of zero may indicate that no resolved rural cases contribute a recorded duration rather than instantaneous resolution.

4.4 Streamlit interface

The Streamlit application provides a single public interface with two workspaces selected from the sidebar. The customer workspace includes quick actions for outage reporting, billing information, prepaid-token checking and support-case follow-up. The manager workspace presents deterministic KPI cards, category/status charts, unresolved cases and the Mkurugenzi conversation interface.

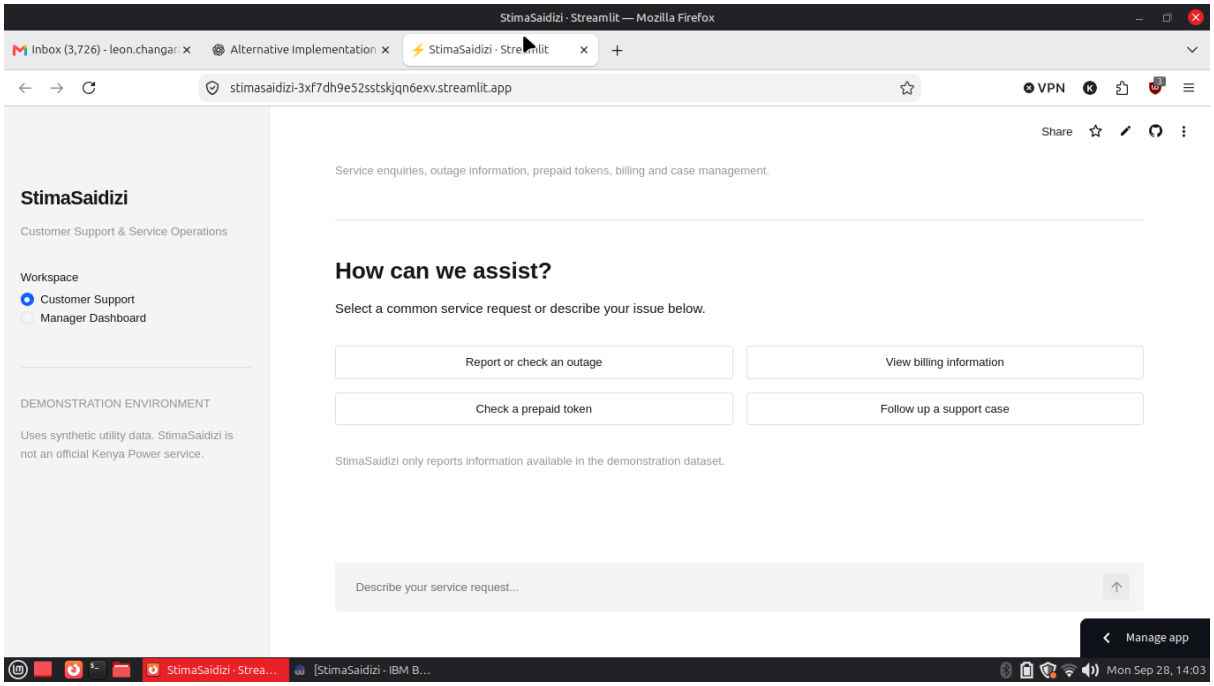


Figure 3: Deployed Customer Support workspace on Streamlit Community Cloud.

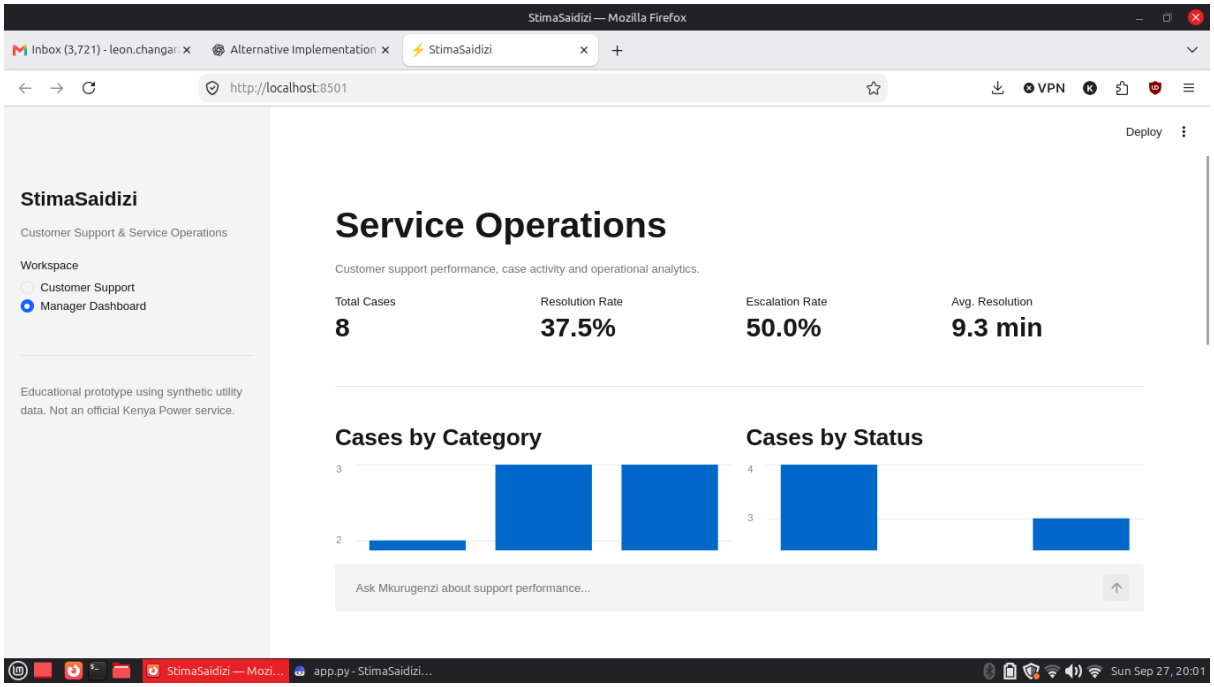


Figure 4: Service Operations dashboard showing support KPIs and category/status distributions.

5 Architecture Evolution

5.1 Original IBM design

The original project plan specified watsonx.ai and Watson Orchestrate as the primary AI technologies. The proposed workflow used Watson Orchestrate to route enquiries and invoke application tools, with watsonx.ai supporting language understanding, summaries and explanations. The plan also recommended Python and FastAPI for deterministic application logic, JSON/SQLite/CSV for synthetic MVP data, Streamlit or a lightweight web interface, and an emphasis on reliable end-to-end workflow over unnecessary infrastructure.

Initial IBM integration artifacts and Orchestrate-oriented documentation were preserved in the repository as an implementation baseline.

5.2 Migration of the orchestration layer

During implementation, access, model availability and deployment constraints made the original IBM path less reliable for completing the end-to-end demonstration. Because the project had already separated orchestration from service logic, the conversational layer could be migrated without replacing the core backend.

Google ADK was selected as the replacement agent framework. Gemini access was tested, but model availability issues led to an external model-access path through LiteLLM and Groq. The final agents use GPT-OSS through Groq, with LiteLLM providing the model abstraction.

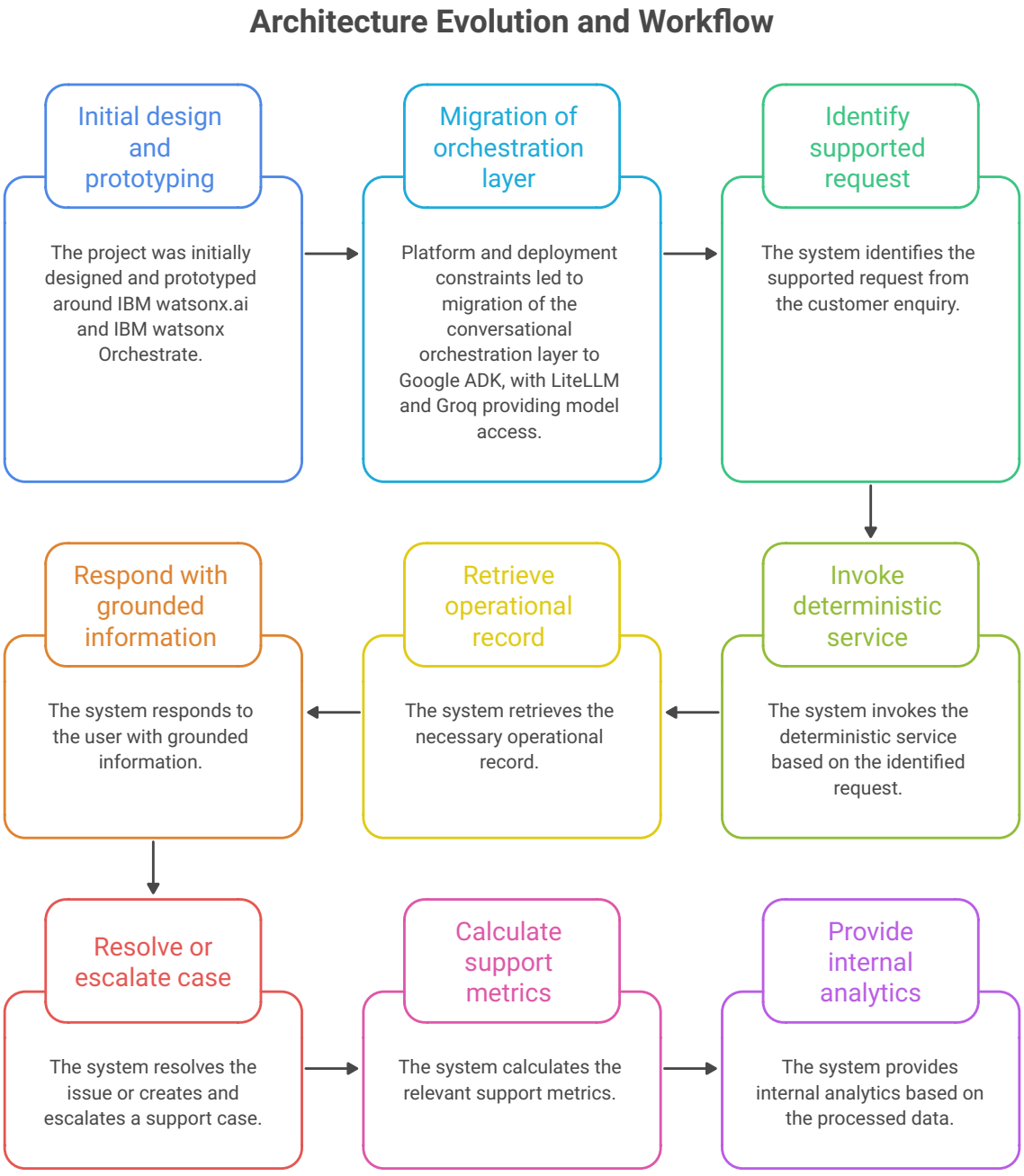


Figure 5: Architecture evolution from the IBM-centred design to the final orchestration stack.

The migration demonstrates the value of modular architecture: the user interface, deterministic services, synthetic datasets, API routes and tests remained useful while only the AI/orchestration layer changed substantially.

6 Testing and Validation

6.1 Automated service tests

The deterministic service suite was executed using `pytest -q`. The final verification completed with 13 tests passing. These tests cover the customer, outage, token, billing and case-service behaviour used by the agents.

```
............. [100%]
13 passed in 0.49s
```

6.2 Live customer-support validation

The deployed application was manually tested against representative scenarios.

Scenario	Observed behaviour	Validation point
Prepaid token: TXN002	Agent requested the transaction reference, reported successful payment and pending token issuance.	Correct distinction between payment and token status; no invented token number or waiting time.
Billing: CUS003	No bill was found; agent offered support-case creation.	Missing data handled explicitly instead of fabricating a bill.
Case creation	Billing issue created CASE009 with category billing and status open.	Case ID and status originated from backend case creation.
Outage workflow	Recorded outage information was returned without inventing an updated restoration estimate when the stored estimate had passed.	Grounded outage handling.

Table 4: Representative live customer-support validation scenarios.

6.3 Mkurugenzi validation

Mkurugenzi was asked to summarise current support operations. For the baseline dataset it reported eight cases, a 37.5% resolution rate, a 50% escalation rate and an average resolution time of 9.33 minutes. It also explained that the rural value of 0.0 minutes reflected the absence of resolved rural cases with a recorded resolution time rather than instant resolution.

The agent additionally stated that performance trends could not be concluded without historical comparison. This behaviour is consistent with the project's requirement that the analytics assistant explain calculated values without inventing trends or causal claims.

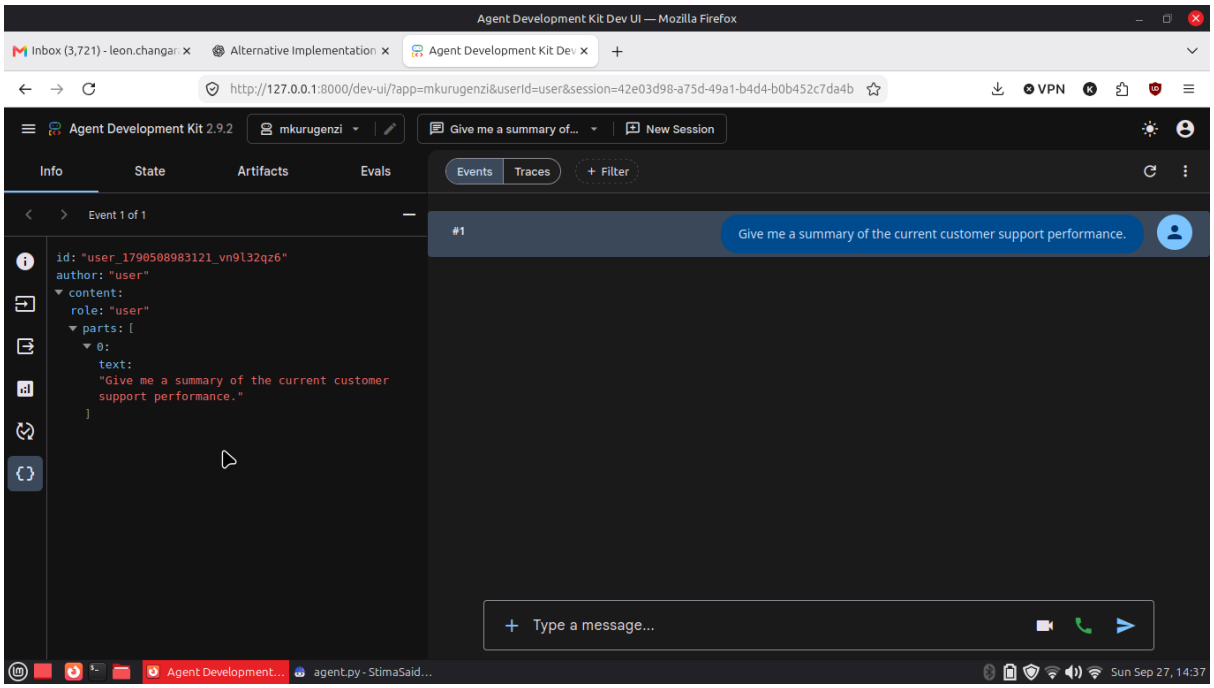


Figure 6: Mkurugenzi during ADK development validation.

6.4 Cross-feature validation

The strongest end-to-end test created CASE009 through Customer Support and then verified that the case became available to the manager-side unresolved-case workflow without restarting the application. This demonstrates that Customer Support and Internal Analytics are connected through shared application data rather than operating as independent chatbot demonstrations.

7 Deployment

7.1 Container validation

A Docker configuration was added for reproducible container deployment. The application image was built successfully and the Streamlit server was run locally on port 8080 with the required model credential supplied through an environment file. This confirmed that the application could run inside a container independently of the local development server.

7.2 Cloud deployment path

Google Cloud Run was evaluated as a deployment target. A Google Cloud project was created and the Google Cloud CLI configured, but the account required a billing prepayment before the intended Cloud Run path could proceed. Because the Cornerstone objective was a reliable demonstration rather than infrastructure expenditure, the project was deployed instead through Streamlit Community Cloud.

The final public application is available at:

https://stimasaidizi-3xf7dh9e52sstskjqn6exv.streamlit.app/

The deployment uses platform-managed secrets for the Groq API credential. The credential is not stored in the repository.

8 Security, Governance and Responsible AI

The original project plan identified regulatory and governance considerations including the Kenya Data Protection Act, ODPC obligations, consumer-protection requirements, human escalation, auditability and prevention of fabricated operational information. The Cornerstone implementation does not claim production regulatory compliance; instead, it applies these principles at prototype level and identifies them as requirements for any future real-world deployment.

8.1 Implemented safeguards

- Synthetic customer and operational data are used throughout the prototype.
- Environment and secret files are excluded from source control.
- Customer-facing operational answers are grounded in service-tool results.
- The agent is instructed not to fabricate outage, billing, token or support-case information.
- Customer and manager functions are separated into distinct workspaces and agents.
- Management metrics are calculated deterministically before AI explanation.
- Unsupported trends and causal claims are explicitly avoided in Mkurugenzi instructions.

8.2 Production requirements

A production deployment would require substantially stronger controls, including customer identity verification, role-based authorisation, encrypted persistent storage, comprehensive audit logging, retention policies, production incident handling, secure integration with utility systems, and formal regulatory/privacy review.

9 Results and Discussion

The project successfully demonstrates the two selected Cornerstone audience features through a single shared operational flow. The customer-facing agent can retrieve structured information and create support cases, while the manager-facing workspace can analyse the same case dataset. This is more meaningful than presenting two unrelated conversational demos because the internal analytics feature reflects changes produced by the customer-support workflow.

The deterministic-service approach also addresses a central reliability problem in operational AI systems. The model is used for language and interaction, but values such as transaction status, billing data, case identifiers and support metrics are obtained from tools. The live TXN002 test illustrates this distinction: the system correctly reported a successful payment with pending token issuance rather than treating payment success as proof that a token had already been generated.

The architecture migration is another significant outcome. Although the final orchestration stack differs from the initial IBM proposal, the project objectives and service architecture remained stable. This demonstrates that the original modular design reduced vendor coupling and allowed the project to adapt to platform constraints while preserving tested application logic.

The final Streamlit interface also improved the demonstrability of the project. Customer and manager workflows are available in one application, with the manager dashboard providing immediate visual grounding before Mkurugenzi is used for natural-language analysis.

10 Limitations and Future Development

10.1 Current limitations

- All customer, outage, billing, token and support-case data are synthetic.
- The application is not connected to live Kenya Power systems.
- There is no real payment processing or prepaid-token generation.
- Production customer authentication and authorisation are not implemented.
- The Streamlit deployment uses prototype file-based data. Runtime case writes are not durable production storage and may be lost when the application instance restarts or redeploys.
- SMS, USSD, WhatsApp and field-service integrations are not implemented.
- The system has not undergone production security, privacy or regulatory certification.

10.2 Future work

Future development can extend the prototype with persistent database storage, authenticated customer and manager roles, approved utility-system integrations, SMS/USSD/WhatsApp channels, authorised payment verification, geospatial outage visualisation, audit/observability infrastructure, expanded multilingual support, service-volume forecasting and human-approved field-service workflows.

A future production design should preserve the project's central principle: consequential operational information should remain grounded in authorised systems of record, with AI acting as the interaction and explanation layer rather than an independent source of operational facts.

11 Conclusion

StimaSaidizi demonstrates an end-to-end AI-assisted electricity customer-support and internal analytics workflow for a Kenyan utility context. The prototype combines a conversational customer assistant, deterministic service tools, structured support-case data, a management dashboard and a grounded analytics copilot.

The final implementation satisfies the central functional objectives of the original Cornerstone plan while adapting the orchestration technology in response to practical platform constraints. Automated tests, live customer scenarios, Mkurugenzi validation and cross-feature case visibility demonstrate that the core workflow operates as intended.

The project also illustrates a broader product-development lesson: modular separation between AI orchestration and deterministic application logic improves reliability, testability and adaptability. For StimaSaidizi, that separation made it possible to preserve the original customer-support and analytics design while moving from an IBM-centred orchestration prototype to a deployable Google ADK, LiteLLM, Groq and Streamlit implementation.

A Repository Structure

```
StimaSaidizi/
|-- agents/
| |-- customer_support/
| `-- mkurugenzi/
|-- api/
| `-- routes/
|-- data/
|-- docs/
| |-- architecture/
```

```
| `-- orchestrate/
|-- frontend/
| `-- app.py
|-- services/
|-- tests/
|-- .streamlit/
|-- Dockerfile
|-- requirements.txt
`-- README.md
```

B Demonstration Script

A concise demonstration can use the following sequence:

1. Open the Customer Support workspace and select *Check a prepaid token*.
2. Enter transaction reference `TXN002`; show the distinction between successful payment and pending token issuance.
3. Start a billing problem for `CUS003`; create a support case and record the generated case ID.
4. Switch to the Manager Dashboard and show the KPI cards, category/status charts and unresolved cases.
5. Ask Mkurugenzi to summarise current support operations.
6. Ask Mkurugenzi to show unresolved cases and demonstrate that the newly created customer case is visible to the manager workflow.
7. Explain the synthetic-data limitation and the non-durable nature of file-based case writes in the hosted prototype.

C Project Artifacts

The repository preserves both the final implementation and selected IBM integration artifacts. Key artifacts include the Streamlit application, ADK agent definitions, deterministic services, FastAPI routes, synthetic datasets, automated tests, Docker configuration, architecture diagrams and Orchestrate documentation retained from the initial implementation path.

D Source Basis for this Report

This report was prepared from the original StimaSaidizi Cornerstone project plan, the implemented repository structure and service behaviour established during development, the final architecture visuals, automated test results, and screenshots/manual validation from the deployed prototype. Where the original plan identified regulatory or production considerations, they are presented here as prototype requirements or future deployment considerations rather than as claims of production compliance.

Demonstration Runbook

The final demonstration can reproduce the same scenarios used during live validation. The sequence below uses only the customer-support, case-management and manager-analytics tests performed on the deployed prototype.

Prepaid-token transaction

Workspace: Customer Support

Prompt: "I bought tokens but haven't received the token number."

When the agent requests the transaction reference, enter **TXN002**. Show that the payment is reported as successful while token issuance remains pending. The key validation point is that the agent distinguishes payment status from token status and does not invent a token number or waiting time.

Billing problem and support-case creation

Workspace: Customer Support

Prompt: "I want to report a billing problem."

When requested, enter customer ID **CUS003**. The agent should report that no bill is available and offer support-case creation. Continue with:

Prompt: "My bill appears incorrect and I would like it reviewed."

During live validation, this workflow created **CASE009** with category *billing* and status *open*. During the final demonstration, use the case ID returned by the backend if a new case is generated.

Mkurugenzi analytics summary

Workspace: Manager Dashboard / Mkurugenzi

Prompt: "Give me a summary of the current customer support performance."

Show that Mkurugenzi explains metrics calculated by the analytics service. In the validated baseline, it reported eight total cases, a 37.5% resolution rate, a 50% escalation rate and an average resolution time of 9.33 minutes. It also correctly explained that a rural value of 0.0 minutes did not mean instant resolution and avoided claiming a performance trend without historical comparison.

Cross-workspace case visibility

After creating the billing case, switch to the Manager Dashboard and inspect the unresolved-case workflow. Demonstrate that the newly created customer case is visible on the manager side. This confirms that Customer Support and Internal Analytics use shared application data rather than operating as independent chatbot demonstrations.

Closing note. State that StimaSaidizi uses synthetic demonstration data and that file-based case writes in the hosted prototype are not durable production storage.